

ExempleCallable (v3)

Fabrice.Kordon@lip6.fr



Objectif de cette version

2

Listes de tâches

-  Insertion dans un pool
-  Attente et récupération des résultats
 - Toujours une somme sur la liste des résultats récupérés

Ha! on commence à jouer?



La classe Calcule

3

```
import java.util.Random;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;
import java.util.ArrayList;
import java.util.Date;

class Calcule implements Callable<Integer> {
    String monNom;
    private Random generateur = new Random();

    Calcule (String n) {
        monNom = n;
    }

    public Integer call() {
        Integer resultat = generateur.nextInt(1000);
        try { // simulation d'un temps de calcul
            Thread.sleep(resultat * 10);
        }
        catch (Exception e) {
            System.out.println("Ne devrait pas arriver " + e);
        } // Affichage après le calcul
        System.out.println(monNom + " rend " + resultat);
        return resultat;
    }
}
```

La «classe principale»

4

```
public class ExempleCallable3 {
    static int nbThreads;
    static int taillePool;

    public static void main(String []args) {
        int somme = 0;

        try {
            nbThreads = Integer.parseInt(args[0]);
            taillePool = Integer.parseInt(args[0]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, nombre de threads, taille du pool.");
            return;
        }

        Integer total = 0;
        ExecutorService executor = Executors.newFixedThreadPool(taillePool);
        ArrayList<Future<Integer>> lesResultats = new ArrayList<Future<Integer>>();
        System.out.println("main : je lance les calculs");
        for (int i = 1; i <= nbThreads ; i++) {
            Future<Integer> f = executor.submit(new Calcule("c"+i));
            lesResultats.add(f);
        }
    }
}
```

La «classe principale»

4



Question...

Et si on oublie le shutdown() ?

```
System.out.println("main : j'attends les résultats");
for (Future<Integer> res : lesResultats) { // Parcourir la collection
    try{
        int x = res.get(); // peut lever InterruptedException
        System.out.println("main : " + x + " reçu à " + new Date());
        somme = somme + x;
    }
    catch (Exception e) {
        e.printStackTrace();
    }
}
System.out.println("main : j'ai calculé " + somme);
executor.shutdown();
}
```

Exécution...

5

```
$ java ExempleCallable3 12 5
main : je lance les calculs
main : j'attends les résultats
c7 rend 107
c5 rend 219
c8 rend 226
c3 rend 282
c9 rend 289
c4 rend 295
c6 rend 346
c12 rend 349
c10 rend 668
c1 rend 746
main : 746 reçu à Wed Oct 05 10:16:10 CEST 2016
c2 rend 782
main : 782 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 282 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 295 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 219 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 346 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 107 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 226 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 289 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 668 reçu à Wed Oct 05 10:16:11 CEST 2016
c11 rend 784
main : 784 reçu à Wed Oct 05 10:16:11 CEST 2016
main : 349 reçu à Wed Oct 05 10:16:11 CEST 2016
main : j'ai calculé 5093
```



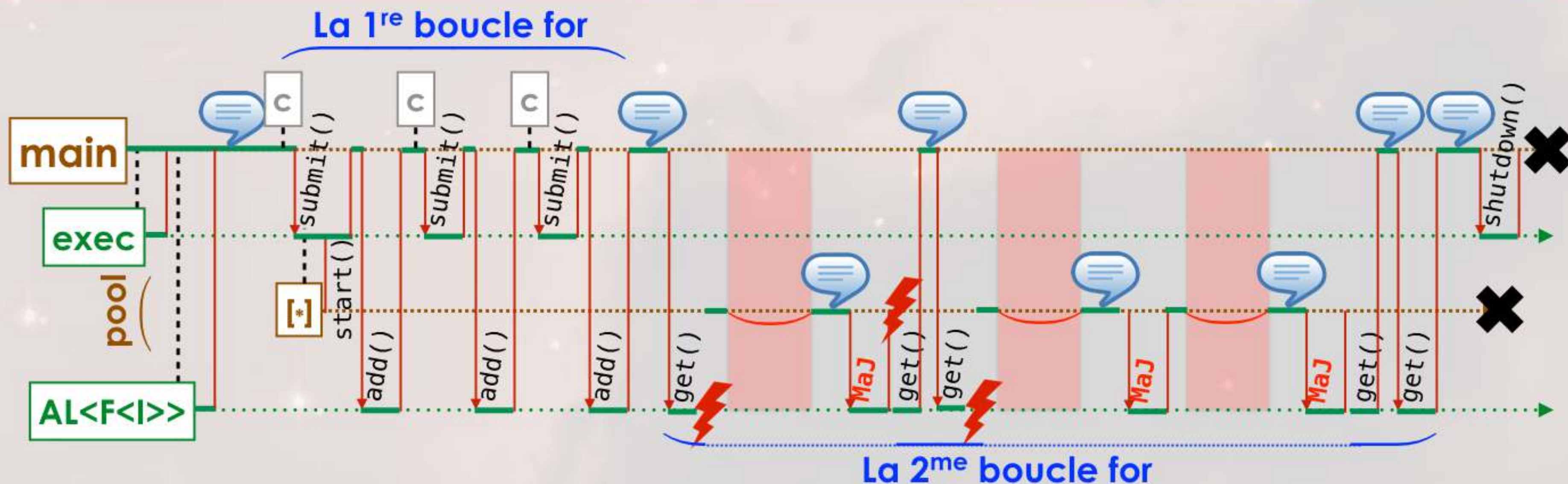
Observation

c1 et c2 ralentissent fortement
l'extraction des résultats



Un petit chronogramme pour la route

6



```
$ java ExempleCallable3 3 1
main : je lance les calculs
main : j'attends les résultats
c1 rend 650
main : 650 reçu à Fri Oct 30 17:13:58 CET 2015
c3 rend 784
c2 rend 784
main : 784 reçu à Fri Oct 30 17:14:01 CET 2015
main : 784 reçu à Fri Oct 30 17:14:01 CET 2015
main : j'ai calculé 2218
```